

What's built, what's left, and the order to do it in

The app looks finished and the backend is now on Firebase — but three things stand between this build and real users, and several screens work only because nobody is on the other side yet. This is the full picture, a day-by-day plan to deployment, and a 41-item checklist to tick off as you go.

24 SCREENS BUILT	34 COMPONENTS	12 SERVICES	3 LANGUAGES + RTL	3 HARD BLOCKERS	0 TESTS	41 OPEN TASKS
---------------------	------------------	----------------	-------------------------	--------------------	------------	------------------

DONE	works end to end	PARTIAL	works, but not for two real users	STUB	UI only — nothing behind it	MISSING	not started
------	------------------	---------	-----------------------------------	------	-----------------------------	---------	-------------

PART ONE

What is actually built

The front end is genuinely complete: 24 screens, a full component library, light and dark palettes, and three languages with right-to-left support for Urdu. As of today the backend runs on Firebase — Auth for sessions, Firestore for data, and security rules written to replace Postgres row-level security. The honest read is that **presentation is finished and plumbing is half-finished**.

AREA	STATE	WHERE IT STANDS
Sign up & login	DONE	Email/password on Firebase Auth, session survives app restart, account deletion clears all seven subcollections.
Onboarding flow	DONE	Welcome → signup → intent & photos → selfie → CNIC, all wired to the profile document.
Profile & edit	DONE	View, edit, rishta profile, dating/rishta mode toggle.
Discovery deck	DONE	Reads real members from Firestore, gender-filtered, falls back to demo profiles when the collection is empty.
Favourites, like limit, view history, privacy prefs	DONE	All four on Firestore, all owner-scoped.
Theming & localisation	DONE	Light/dark, English + Roman Urdu + Urdu, RTL layout.
Matches & chat	PARTIAL	Full chat UI with a live Firestore listener — but it only ever reads and writes your own copy. See Blocker 1.
Notifications	PARTIAL	In-app feed and preferences work. Nothing arrives when the app is closed.
Block list	PARTIAL	Blocking is recorded on your side; it does not yet filter the other person out of any query.

AREA	STATE	WHERE IT STANDS
Security rules	PARTIAL	Written and correct for today's model — but never deployed, and they will need rewriting for Blocker 1.
Media upload	STUB	Deliberate placeholder. See Blocker 2.
Explore Plus subscription	STUB	Tapping upgrade flips a boolean. No payment of any kind.
Audio & video calls	STUB	Camera preview and a running timer. No call is placed.
Events in Explore	STUB	Still reading the hard-coded demo list.
Wali (guardian) dashboard	STUB	Invite is stored locally; no guardian account or delivery exists.
Report a user	STUB	Shows a confirmation toast and discards the report.
Privacy policy & terms	STUB	Placeholder text — the screen says so itself.
Password reset, email verification	MISSING	Neither exists. A forgotten password currently locks the user out permanently.
Push notifications	MISSING	expo-notifications is not installed.
Automated tests	MISSING	No test file in the repo.

Three blockers

Nothing else on the list matters until these are done. Each one is the reason a feature that looks finished would fail the moment two real people used the app at the same time.

01 Likes, matches and messages never reach the other person

What's wrong. Every write goes into your own private tree — `users/{uid}/matches` and `users/{uid}/messages`. When you like someone, a match document is created *for you*. When you send a message, it is saved *to your own* collection. The other user's app has no way to see any of it, and the security rules would refuse the write anyway, because everything under `users/{uid}` is owner-only by design.

What it means today. The chat screen is effectively a notes app. Two phones signed into two accounts will never see each other. “It's a match” fires off a local favourite toggle, not a mutual like.

The fix. Introduce data that belongs to *both* people rather than one:

- `likes/{fromUid}_{toUid}` — one document per like. A match exists when the mirrored document also exists.
- `conversations/{convId}` with a `participants: [uidA, uidB]` array, and `conversations/{convId}/messages/{id}` underneath it.
- Rules rewritten so a document is readable and writable by anyone listed in `participants` — not just by a single owner.
- The existing live listener moves from your own messages collection onto the shared conversation, which is a small change once the model is right.

The services keep their current exported functions, so screens do not need rewriting — this is a data-model and rules job, not a UI job.

02 No photo, voice or video actually leaves the phone

What's wrong. Firebase Cloud Storage needs the paid Blaze plan, so `mediaUpload.ts` was left as a deliberate placeholder: every upload returns the local device URI unchanged, and that URI is what gets saved to Firestore.

What it means today. Profile photos, voice intros, video intros and chat images render only on the device that picked them — anyone else sees a blank card. Deleting a photo deletes nothing. CNIC scans and selfies are never stored anywhere.

The fix. Choose a backend and rewrite one file. Options: turn on Blaze and use Firebase Storage (rules for it are already written and waiting), or use a free-tier alternative such as Cloudinary, ImageKit or Cloudflare R2. Everything media-related in the app funnels through this single module, so no other file changes.

03 Nothing is deployed and no real data has been migrated

What's wrong. The Firestore and Storage rules exist only as files in the repo.

`firestore.indexes.json` is empty, so the filtered-and-sorted queries the app already makes will start failing with index errors as soon as the collection has real volume. The migration script that moves existing Supabase data across has never been run, not even in dry-run mode. And the entire migration is still sitting uncommitted in the working tree.

The fix. Commit; deploy rules; add the composite indexes Firestore asks for; dry-run the migration, read the skipped/error tally, then run it live. Note the ordering trap: *rules must be live before real data lands*, and the rules will change when Blocker 1 is done — so do the model work first, then deploy once.

PART THREE

Features that look finished but aren't

These are worth listing separately because a demo of the app would not reveal any of them. Each screen behaves convincingly; nothing is connected behind it.

Identity verification STUB Uploading a CNIC photo sets <code>cnicVerified: true</code> on the user's own document. Nobody reviews it, no NADRA check runs, and the selfie step performs no face match or liveness test. Any user can mark themselves verified.	Report a user STUB The report dialog collects a reason, shows “report sent”, and drops it. There is no reports collection, no moderation queue, and no admin tool of any kind.
Explore Plus STUB Upgrading writes <code>isExplorePlus: true</code> and a renewal date to the user's own profile. No in-app purchase, no card, no JazzCash or Easypaisa. Anyone can grant themselves a paid plan for free.	Audio & video calls STUB A camera preview, a ringing animation and a counting timer. There is no WebRTC layer, no signalling, no TURN server. This is a substantial project on its own — strongly consider cutting it from version one.
Wali dashboard STUB A guardian's name and contact are captured and kept locally. No invitation is sent, no guardian account exists, and no guardian can see anything.	Explore events STUB The events tab still maps over the hard-coded demo list. Needs either a Firestore collection an admin can fill, or removal from version one.

PART FOUR

Decisions only you can make

Each of these changes the plan below. They are worth settling before the work starts, not during it.

QUESTION	WHY IT MATTERS	SUGGESTED
Turn on the Blaze plan?	Decides media storage, and whether Cloud Functions are available for push and moderation. Blaze is pay-as-you-go with a free tier, not a flat fee.	Yes — it unblocks three items at once
How do people pay?	Apple and Google require their own in-app purchase for digital subscriptions. A local gateway is only allowed for physical goods and services.	Store IAP, or make v1 free
Calls in version one?	Real calling is a multi-week project with running server costs.	Cut from v1
Events in version one?	Needs an admin tool and someone to keep it filled.	Cut from v1
Who reviews CNICs and reports?	Determines whether an admin panel is needed before launch or after.	Firebase console at first

From here to the store

Six phases, each broken into roughly one working day. Phases are in dependency order — Phase 1 has to come before the rules are deployed, and media has to work before real profiles are worth migrating. Assuming calls and events are cut from version one, this is about **six weeks of focused work** for one developer.

PHASE 1 Make two people connect 7 days	
☐ Day 1	Design the shared model — <code>likes</code> , <code>conversations</code> , <code>participants</code> — and write it into <code>firestorePaths.ts</code> with the layout comment updated.
☐ Day 2	Rewrite <code>firestore.rules</code> for participant-based access; verify every case in the Rules Playground before touching app code.
☐ Day 3	Build <code>likesService</code> : record a like, detect the mirrored like, create the conversation on a mutual match.
☐ Day 4	Point <code>matchesService</code> at conversations, keeping its exported functions unchanged so no screen breaks.
☐ Day 5	Move the live listener in <code>MatchesContext</code> onto the shared conversation and its messages.
☐ Day 6	Two-device test: like, match, send text, voice and image; check unread counts and last-message ordering.
☐ Day 7	Make blocking real — filter blocked users out of discovery and hide their conversations.
PHASE 2 Media that actually uploads 4 days	
☐ Day 1	Decide the backend and set up the account and credentials.
☐ Day 2	Rewrite <code>mediaUpload.ts</code> against it — upload, public URL, delete. One file, no other changes.
☐ Day 3	Keep verification media private and separate from public photos; confirm nobody else can fetch a CNIC scan.
☐ Day 4	Two-device test that photos, voice intros and chat images render for the other person; point the migration script's file copy at the new destination.
PHASE 3 Trust and safety 5 days	
☐ Day 1	Write reports to a <code>reports</code> collection with reporter, target, reason and timestamp.
☐ Day 2	Separate <i>submitted</i> from <i>verified</i> : verification becomes a pending status a reviewer approves, not a flag the user sets.
☐ Day 3	Build the review path — Firebase console views to start, an admin screen later.
☐ Day 4	Add password reset and email verification. This is small and currently locks out anyone who forgets a password.
☐ Day 5	Replace the placeholder privacy policy and terms with reviewed text, and host both at public URLs the stores can reach.
PHASE 4 Push notifications 4 days	
☐ Day 1	Install and configure expo-notifications with FCM and APNs credentials.
☐ Day 2	Save each device's push token to the user document and refresh it on login.
☐ Day 3	Send on new match and new message — a Cloud Function triggered by the write is the natural home.
☐ Day 4	Wire the existing notification preferences so they genuinely mute what they say they mute.

PHASE 5 Payments — skip if v1 ships free		5 days
☐	Day 1	Create the subscription products in App Store Connect and Google Play Console.
☐	Day 2	Integrate in-app purchase in the Explore Plus screen.
☐	Day 3	Validate receipts server-side — never trust a client flag for paid status.
☐	Day 4	Handle renewal, expiry, cancellation and restore-purchases.
☐	Day 5	Test the whole flow in both stores' sandbox environments.

PHASE 6 Ship it		6 days
☐	Day 1	Add every composite index Firestore asks for; deploy Firestore and Storage rules.
☐	Day 2	Dry-run the Supabase migration, read the skipped and error tally, fix, then run it live.
☐	Day 3	Build a preview APK with EAS and put it in front of real testers on real phones.
☐	Day 4	Fix what testing finds — expect a day of nothing but this.
☐	Day 5	Store listings: screenshots, descriptions in English and Urdu, privacy policy link, and the data-safety and privacy-nutrition forms.
☐	Day 6	Production build, submit to both stores, and wait out review.

The checklist

Every open task in one place, in the order the plan runs. Print this page, keep it beside you, and tick a line the day it is genuinely finished — not the day it is started. Groups F and G hold the two traps: **the cut-or-build decisions belong at the very start**, and **nothing gets deployed until the model in group A is settled**.

☐	not started	right-hand column — write the day you finished it	41 tasks total
A · Make two people connect		PHASE 1 · 7 DAYS	
☐	01	Design the shared model — <code>likes/</code> and <code>conversations/</code> with a <code>participants</code> array — and write it into <code>firebasePaths.ts</code> .	
☐	02	Rewrite <code>firebase.rules</code> for participant-based access; verify every case in the Rules Playground first.	
☐	03	Build <code>likesService</code> — record a like, detect the mirrored like, create the conversation on a match.	
☐	04	Point <code>matchesService</code> at conversations, keeping its exported functions unchanged.	
☐	05	Move the live listener in <code>MatchesContext</code> onto the shared conversation and its messages.	
☐	06	Two-device test — like, match, send text, voice and image; check unread counts and ordering.	
☐	07	Make blocking real — filter blocked users out of discovery and hide their conversations.	
B · Media that actually uploads		PHASE 2 · 4 DAYS	
☐	08	Decide the media backend — Blaze Storage or a free-tier alternative — and set up the account.	
☐	09	Rewrite <code>mediaUpload.ts</code> against it: upload, public URL, delete. One file, no other changes.	
☐	10	Keep verification media private and separate; confirm nobody else can fetch a CNIC scan.	
☐	11	Two-device media test; point the migration script's file copy at the new destination.	
C · Trust and safety		PHASE 3 · 5 DAYS	
☐	12	Write reports to a <code>reports</code> collection with reporter, target, reason and timestamp.	
☐	13	Make verification a pending status a reviewer approves — not a flag the user sets on themselves.	
☐	14	Build the review path — Firebase console views to start with, an admin screen later.	
☐	15	Add password reset. Small job, and right now a forgotten password is permanent.	
☐	16	Add email verification.	
☐	17	Replace the placeholder privacy policy and terms, and host both at public URLs.	
D · Push notifications		PHASE 4 · 4 DAYS	
☐	18	Install and configure expo-notifications with FCM and APNs credentials.	
☐	19	Save each device's push token to the user document and refresh it on login.	
☐	20	Send on new match and new message — a Cloud Function triggered by the write.	
☐	21	Wire the existing notification preferences so they genuinely mute what they say they mute.	

E · Payments — skip if v1 ships free		PHASE 5 · 5 DAYS
☐	22 Create the subscription products in App Store Connect and Google Play Console.	
☐	23 Integrate in-app purchase in the Explore Plus screen.	
☐	24 Validate receipts server-side — never trust a client flag for paid status.	
☐	25 Handle renewal, expiry, cancellation and restore-purchases.	
☐	26 Test the whole flow in both stores' sandbox environments.	
F · Cut or build — decide before Phase 1 starts		DECISION
☐	27 Events — back the tab with a real Firestore collection an admin can fill, or remove it from v1.	
☐	28 Wali dashboard — build a real invite and guardian view, or remove it from v1.	
☐	29 Audio and video calls — commit to WebRTC and a TURN server, or remove them from v1.	
G · Backend and deploy		PHASE 6 · 6 DAYS
☐	30 Commit the branch. The whole Firebase migration is still sitting uncommitted in the working tree.	
☐	31 Add every composite index Firestore asks for — <code>firestore.indexes.json</code> is empty today.	
☐	32 Deploy <code>firestore.rules</code> and <code>storage.rules</code> — before any real data lands, not after.	
☐	33 Dry-run the Supabase migration and read the skipped and error tally.	
☐	34 Fix what the dry run surfaced, then run the migration live.	
☐	35 Add tests for matching, messaging and payment logic — the three places a silent bug costs real money or trust.	
H · Out the door		RELEASE
☐	36 Build a preview APK with EAS and put it in front of real testers on real phones.	
☐	37 Fix what testing finds — expect a full day of nothing but this.	
☐	38 Screenshots for both stores, at every size they demand.	
☐	39 Store descriptions in English and Urdu, with the privacy policy link.	
☐	40 Fill in the Play data-safety form and the App Store privacy nutrition labels.	
☐	41 Production build, submit to both stores, and wait out review.	

WORTH KNOWING

Two things that will bite later

- **There are no tests at all.** That is survivable for the UI, but not for matching, messaging and payment logic. Add tests for those three as you build them in Phases 1 and 5 — retrofitting them after launch is far more expensive.
- **Dating apps get extra scrutiny in review.** Both stores look hard at age verification, reporting and blocking, and the privacy policy. Phase 3 is not optional polish — it is what gets the app approved.